



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Modern Compiler Design

Subject Code – BE103701

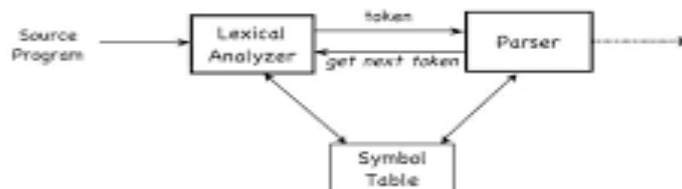
Faculty Name – Radhika Gokani

CHAPTER -3

3. Lexical Analysis:

Lexical analysis is the **first phase of a compiler**, where raw source code is transformed into a stream of **tokens**—the building blocks of syntax. Think of it as the compiler’s way of learning to “read” code by breaking it down into understandable pieces like keywords, identifiers, operators, and punctuation.

Lexical Analysis



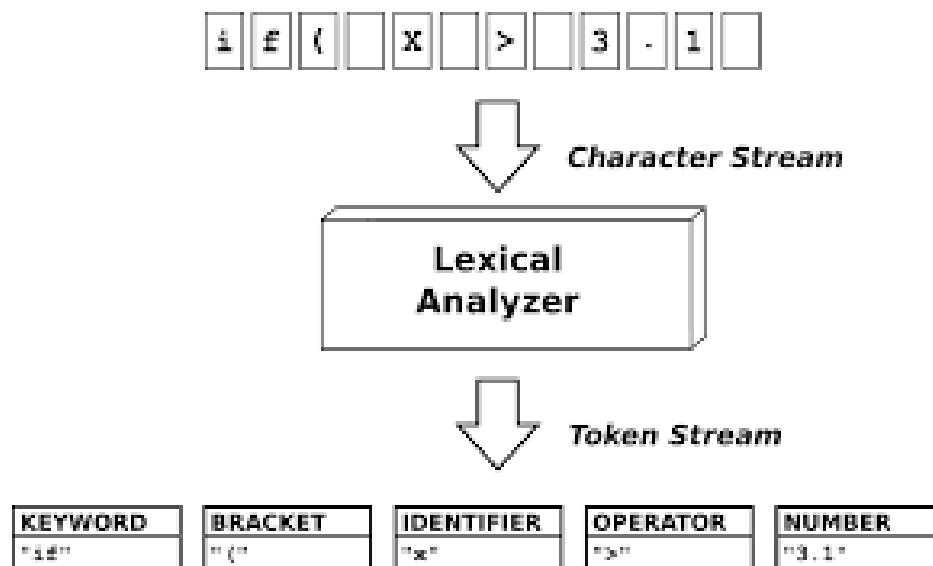
Lexical Analysis Step-by-Step

1. **Understand the Basics** Introduction to Lexical Analyzer explains the core functions of a lexical analyzer, including token classification and DFA transitions. It’s a great starting point if you’re new to compiler design.
2. **See It in Action with Examples** Lec-3: Lexical Analysis in Compiler Design with Examples walks through tokenization and error handling with practical examples. This helps you visualize how code is broken down.
3. **Explore Real-World Applications** What is Lexical Analysis? | Role of Lexical Analyzer dives into how lexical analyzers handle errors and recover from them, which is crucial for building robust compilers.

4. **Practice with Solved Problems** Lexical Analyzer (Solved Problems) - Set 1 features GATE-level questions and solutions, ideal for exam prep or sharpening your problem-solving skills.
5. **Learn in Hindi** Lexical Analysis in Compiler Design | In Hindi | Easy ... offers a clear and accessible explanation for Hindi-speaking learners.
6. **Use Tools Like Flex** Lexical Analysis using Flex shows how to automate lexical analysis using Flex, a powerful tool that converts regular expressions into C code for token recognition.

3.1 The Role of the Lexical Analyzer

The **lexical analyzer** plays a crucial role as the first phase of a compiler. It acts like a gatekeeper, scanning raw source code and converting it into a structured stream of **tokens**—the basic units of syntax like keywords, identifiers, operators, and punctuation. This process sets the stage for deeper analysis by the parser and other compiler components.



Responsibilities of the Lexical Analyzer

1. **Tokenization** It reads the source code character by character and groups them into meaningful sequences called *tokens*.
2. **Removing Noise** It strips out comments and unnecessary whitespace, simplifying the input for the parser.
3. **Error Detection** It identifies invalid characters or malformed tokens early in the compilation process.
4. **Symbol Table Interaction** When identifiers are found, the analyzer updates the symbol table with relevant information.
5. **Interface with the Parser** The parser requests tokens from the lexical analyzer as needed, making it a dynamic interaction.
6. **Macro Expansion (Optional)** In some compilers, the lexical analyzer also expands macros before passing tokens to the parser.
7. **Line Tracking for Error Reporting** It keeps track of line numbers to associate errors with their location in the source code.

3.2 Specification of Tokens

The **specification of tokens** is a foundational concept in compiler design, particularly in the lexical analysis phase. It defines how the compiler recognizes and categorizes the smallest meaningful units of code—called **tokens**—using formal methods like **strings**, **languages**, and **regular expressions**

Components of Token Specification

1. **Strings**

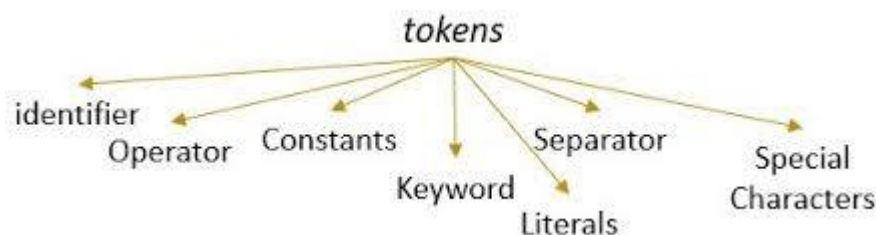
- A string is a finite sequence of characters drawn from a defined alphabet.
- Operations include **prefix**, **suffix**, **substring**, and **concatenation**.

2. **Languages**

- A language is a set of strings over a fixed alphabet.
- It helps define which combinations of characters are valid tokens.

3. **Regular Expressions**

- These are used to define patterns that match valid tokens.
- For example, a regular expression for an identifier might be `[a-zA-Z_][a-zA-Z0-9_]*`.



What is a Lexical Token?

A lexical token is a sequence of characters with a collective meaning in the grammar of programming languages. These tokens include **Keyword**, **Identifier**, **Operator**, **Literal**, and **Punctuation**.

For Example, the following are some lexical tokens:

- **Keywords:** int, String, long, etc.
- **Identifier :** x, y, i, j, num etc.
- **Operators :** +, -, *, / etc.
- **Literals:** 108, 9, 12, 15 etc.
- **Punctuations:** , ; . etc

3.3 Recognition of Tokens

Recognition of tokens is a key function of the **lexical analyzer** in compiler design. It's the process of identifying valid sequences of characters—called **lexemes**—and classifying them into **tokens** like keywords, identifiers, operators, and literals. This step ensures that the compiler can interpret the source code correctly and efficiently.

Recognizing tokens accurately is essential for:

- Ensuring syntactic correctness.
- Feeding structured input to the parser.
- Detecting lexical errors early in the compilation process.

Token Recognition Works

1. **Pattern Matching with Regular Expressions** Each token type (e.g., identifier, number, keyword) is defined by a regular expression. The lexical analyzer scans the input and matches substrings against these patterns.
2. **Transition Diagrams and Tables** These are visual and tabular representations of how the analyzer moves through states to recognize tokens.
3. **Longest Match Rule** When multiple patterns match, the analyzer selects the longest valid lexeme to ensure accurate tokenization.
4. **Error Handling** If no valid token is found, the analyzer flags an error and may attempt recovery.

3.4 Input Buffering

Input buffering is a technique used in **compiler design**, especially during **lexical analysis**, to improve performance by reading input in larger chunks rather than one character at a time. This reduces the number of expensive system calls and speeds up token recognition.

- Instead of reading one character at a time from the source code, the compiler reads a **block of characters into a buffer**.
- Two pointers—**lexemeBegin** and **forward**—are used to track the start and end of the current lexeme.
- When the buffer is exhausted, the next block is loaded, often using **sentinels** to mark buffer boundaries.

Types of Input Buffering

1. **Single Buffer Scheme**
 - Simple but inefficient for large inputs.
 - Requires frequent reloading and pointer resets.
2. **Double Buffer Scheme**
 - Uses two halves of a buffer.
 - While one half is being processed, the other is being filled.
 - Reduces overhead and allows for **lookahead** in token recognition.

Requirement of Input Buffering

- **Performance Boost:** Fewer system calls mean faster compilation.
- **Efficient Lexeme Recognition:** Enables lookahead and backtracking.
- **Memory Management:** Optimizes how input is stored and processed.

3.5 Elementary scanner design and its implementation (Lex)

Designing an elementary scanner and implementing it using **Lex** is a foundational exercise in compiler construction. Lex is a powerful tool that automates the creation of lexical analyzers—programs that read source code and break it into tokens.

An elementary scanner:

- Reads a stream of characters from source code.
- Identifies valid tokens like keywords, identifiers, operators, and literals.
- Ignores whitespace and comments.
- Passes tokens to the parser for further analysis.

Implementation

Lex simplifies this by allowing you to define **patterns** (using regular expressions) and **actions** (code to execute when a pattern is matched).

Structure of a Lex Program

A Lex program is divided into **three sections**, separated by `%%`:

1. **Declarations** Define variables, headers, and macros.
2. **Translation Rules** Each rule has a pattern and an associated action. Example:
[0-9]+ { printf("NUMBER\n"); }
[a-zA-Z]+ { printf("IDENTIFIER\n"); }
3. **Auxiliary Procedures** Additional C functions used in actions.

Lex Works in Practice

1. Write your Lex source code in a .l file.
2. Run the Lex compiler:
lex filename.l
gcc lex.yy.c -o scanner
./scanner < input.txt
3. The scanner reads input.txt, matches patterns, and executes actions.

Lex is a game-changer for compiler design—it abstracts away the complexity of manually writing scanners and lets you focus on defining token patterns.

3.6 Applying concepts of Finite Automata for recognition of tokens

Applying **Finite Automata (FA)** to recognize tokens is a cornerstone of lexical analysis in compiler design. It transforms abstract regular expressions into concrete machines that can scan and classify input strings efficiently.

Finite Automata Helps in Token Recognition

1. **Regular Expressions → Finite Automata** Each token type (e.g., identifier, number, keyword) is defined using a regular expression. These are then converted into finite automata—either **Deterministic (DFA)** or **Non-Deterministic (NFA)**—to recognize valid lexemes.
2. **State Transitions** As the input string is read character by character, the automaton transitions between states. If it reaches a final (accepting) state, the string is recognized as a valid token.
3. **Efficiency and Accuracy** DFAs are particularly useful because they guarantee a single path for each input, making token recognition fast and predictable.

Finite Automata Recognize Tokens

Lexical analyzers use **Deterministic Finite Automata (DFA)** to scan source code and identify tokens. The process:

- **Step 1: Define Token Patterns** Each token type is described using a **regular expression**. For example, an identifier might be `[a-zA-Z_][a-zA-Z0-9_]*`.
- **Step 2: Convert Regex to FA** These regular expressions are converted into **Finite Automata**—either **NFA** (Nondeterministic) or **DFA** (Deterministic). DFAs are preferred for implementation because they are faster and easier to program.

- **Step 3: Scan Input** The DFA reads the input character by character. When it reaches a **final state**, it recognizes a token.
- **Step 4: Emit Token** The recognized token is passed to the next phase of the compiler.

DFA vs NFA

Feature	DFA	NFA
Transitions	One per input symbol	Multiple possible transitions
ϵ -moves (empty transitions)	Not allowed	Allowed
Used in Lexical Analysis	✓ Yes	✗ Mostly theoretical

Example: Recognizing an Identifier

Let's say we want to recognize a C-style identifier:

- **Regex:** `[a-zA-Z_][a-zA-Z0-9_]*`
 - **DFA Logic:**
 - Start state accepts a letter or `_`
 - Loop state accepts letters, digits, or `_`
 - Final state reached when a non-matching character is found
-